

AI FitTrack API — 20-Page Project Content

The uploaded project document identifies **AI FitTrack API** as an **AI-Augmented Backend Development** project. It also provides a project document and reference video, although the linked Google Doc itself was not accessible from here.

Below is a **20-page assignment/project-report format** you can use. Since the supplied file does not contain the detailed FitTrack specification, the implementation details below are a structured project design rather than claims about the inaccessible reference document.

Page 1 — Introduction

AI FitTrack API

AI FitTrack API is a backend application designed to support fitness and wellness tracking through a set of RESTful APIs. The main purpose of the system is to allow users to maintain fitness-related information such as personal profiles, daily activities, exercises, goals, and progress.

An API, or Application Programming Interface, provides a communication layer between the client application and the backend server. In a FitTrack system, a mobile application or web application can send requests to the backend, and the backend processes those requests and returns appropriate responses.

The system can be designed with AI-assisted features to provide personalized fitness insights based on the information supplied by the user. For example, the system may analyze activity records and provide general recommendations or progress summaries.

The backend is responsible for authentication, data validation, database operations, business logic, API routing, and error handling.

Objectives

The major objectives are:

1. To develop a structured fitness-tracking backend.
 2. To provide REST API endpoints.
 3. To securely manage user information.
 4. To store exercise and activity records.
 5. To maintain fitness goals.
 6. To track user progress.
 7. To provide personalized insights.
 8. To validate incoming data.
 9. To implement authentication and authorization.
 10. To create a scalable backend architecture.
-

Page 2 — Problem Statement

Many people record fitness information using different applications or manual methods. When information is distributed across different systems, it becomes difficult to maintain a consistent record of activities and

progress.

A centralized fitness-tracking backend can solve this problem by providing a common platform where users can manage their fitness information.

The FitTrack API addresses the following problems:

- Lack of centralized fitness information.
- Difficulty maintaining daily activity records.
- Difficulty monitoring goals.
- Lack of structured progress information.
- Repeated manual calculations.
- Difficulty connecting a fitness application to a database.
- Need for secure user authentication.
- Need for APIs that can be consumed by web or mobile applications.

The API-based approach separates the frontend from the backend. This allows different client applications to communicate with the same backend services.

Page 3 — Project Objectives

The project is designed around several important objectives.

1. User Management

The application should allow users to create accounts and maintain their profiles.

2. Authentication

Users should be authenticated before accessing protected resources.

3. Activity Tracking

The backend can maintain records of activities such as walking, running, cycling, or other exercises.

4. Exercise Management

Exercise information can be stored and retrieved through API endpoints.

5. Goal Management

Users can create fitness goals and monitor their progress.

6. Progress Tracking

The system can summarize activity and goal progress.

7. AI-Assisted Insights

The backend can use user-provided activity information to generate personalized insights.

8. Data Security

User information should be protected through authentication, authorization, validation, and secure storage.

Page 4 — System Architecture

A typical AI FitTrack API can follow a layered backend architecture.

Main Components

Client → API Routes → Controllers → Services → Database

The client may be a web application, mobile application, or another API consumer.

Client Layer

The client sends HTTP requests such as:

- GET
- POST
- PUT/PATCH
- DELETE

API Layer

The API receives requests and identifies the required operation.

Controller Layer

Controllers process HTTP requests and responses.

Service Layer

The service layer contains the application's business logic.

Database Layer

The database stores users, activities, exercises, goals, and other information.

AI Layer

An AI service can analyze relevant fitness data and return an insight or recommendation.

This layered architecture makes the project easier to maintain and extend.

Page 5 — Technologies Used

A possible technology stack for the FitTrack backend includes:

Backend

A server-side framework such as:

- Node.js
- Express.js

Node.js allows JavaScript to be used for server-side programming. Express.js can be used to create REST API routes.

Database

A database such as:

- MongoDB
- PostgreSQL
- MySQL

can be used depending on the project requirements.

Authentication

Authentication can be implemented using:

- JWT
- Password hashing
- Protected routes

API Testing

Tools such as Postman can be used to test API requests and responses.

Development Tools

Common development tools include:

- Visual Studio Code
- Git
- GitHub
- npm

The exact technology stack should be aligned with the project specification when implementing the actual application.

Page 6 — REST API

REST stands for **Representational State Transfer**.

A REST API allows applications to communicate using HTTP requests.

HTTP Methods

Method	Purpose
GET	Retrieve data
POST	Create data
PUT	Update data
PATCH	Partially update data
DELETE	Delete data

For example:

```
GET /api/users
```

can retrieve users.

```
POST /api/activities
```

can create an activity record.

REST APIs generally return data in JSON format.

Example Response

```
{
  "success": true,
  "message": "Activity created successfully"
}
```

REST architecture makes the backend reusable for different frontend applications.

Page 7 — User Authentication

Authentication is an important component of FitTrack.

A user first creates an account using registration information.

Registration Process

1. User submits registration details.
2. Backend validates the information.
3. Password is securely hashed.
4. User information is stored.
5. Registration response is returned.

Login Process

1. User submits email and password.
2. Backend searches for the user.
3. Password is verified.
4. Authentication token is generated.
5. Token is returned to the client.

JWT

JSON Web Token can be used for authentication.

A protected request can contain:

Authorization: Bearer <token>

The server verifies the token before allowing access to protected resources.

Page 8 — User Profile Management

A FitTrack user profile may contain information such as:

- Name
- Email
- Age
- Height
- Weight
- Activity preference
- Fitness goals

The backend should validate profile information before storing it.

Example Endpoint

GET /api/profile

retrieves the current user's profile.

PUT /api/profile

updates profile information.

Benefits

Profile management allows the application to maintain user-specific information and use appropriate data when generating progress summaries.

Sensitive information should be handled carefully and should only be accessible to authorized users.

Page 9 — Exercise Management

Exercise management is another important component.

The backend can maintain information about different exercises.

Example fields:

```
exerciseId
name
category
description
duration
difficulty
```

Example Exercise Categories

- Cardio
- Strength
- Flexibility
- Balance
- General fitness

Example API

```
GET /api/exercises
```

retrieves available exercises.

```
GET /api/exercises/:id
```

retrieves a specific exercise.

Exercise information can help users select activities suitable for their fitness goals.

Page 10 — Activity Tracking

Activity tracking allows users to record completed activities.

A record may include:

```
activityId
userId
activityType
duration
date
distance
notes
```

For example, a user may record a walking activity.

API Operations

```
POST /api/activities
```

creates an activity.

```
GET /api/activities
```

retrieves activities.

```
GET /api/activities/:id
```

retrieves one activity.

```
DELETE /api/activities/:id
```

deletes an activity.

The backend should verify that the activity belongs to the authenticated user before allowing modifications.

Page 11 — Fitness Goals

Goals help users define targets.

Examples include:

- Activity frequency
- Running distance
- Exercise duration
- General fitness targets

A goal could contain:

```
goalId
userId
title
description
target
currentValue
startDate
endDate
status
```

Goal Status

A goal may have statuses such as:

```
ACTIVE
COMPLETED
CANCELLED
```

The backend can calculate progress based on activity records.

Example

If the target is 100 activity units and the current value is 60:

```
Progress = 60 / 100 × 100
          = 60%
```

The API can return the calculated progress to the frontend.

Page 12 — Progress Tracking

Progress tracking provides users with a summary of their activities over time.

The backend can calculate:

- Total activities
- Total duration
- Total distance
- Completed goals
- Current goals
- Activity frequency

A progress endpoint might be:

```
GET /api/progress
```

Example response:

```
{
  "totalActivities": 15,
  "totalDuration": 450,
  "goalsCompleted": 2
}
```

Progress information can help users understand their activity history.

The backend can also provide date-based filtering:

```
GET /api/progress?from=2026-09-01&to=2026-09-30
```

Page 13 — AI Features

The AI component is one of the distinguishing features of an AI-Augmented FitTrack API.

AI can be used to analyze user-provided fitness information and generate general insights.

Possible features include:

Activity Summary

The system can summarize recent activity.

Goal Insights

The system can identify whether activity records are progressing toward a user-defined goal.

Personalized Suggestions

The system may generate general suggestions based on the user's recorded activities.

Natural Language Interaction

A user could submit a question such as:

How has my activity changed this week?

The backend can gather relevant records and send appropriate structured information to an AI component.

AI output should be treated as informational rather than as professional medical advice.

Page 14 — Database Design

The database stores information required by the application.

Possible entities include:

User

```
User
├── id
├── name
├── email
├── password
└── createdAt
```

Activity

```
Activity
├── id
├── userId
├── activityType
├── duration
├── date
└── notes
```

Goal

```
Goal
├── id
├── userId
├── title
├── target
├── currentValue
└── status
```

Exercise

```
Exercise
├── id
├── name
├── category
└── description
```

A relationship can exist between a user and their activities and goals.

Page 15 — API Endpoint Design

A possible API structure is:

Authentication

```
POST /api/auth/register
POST /api/auth/login
```

Users

```
GET /api/profile
PUT /api/profile
```

Exercises

```
GET /api/exercises
GET /api/exercises/:id
```

Activities

```
POST /api/activities
GET /api/activities
GET /api/activities/:id
PUT /api/activities/:id
DELETE /api/activities/:id
```

Goals

```
POST /api/goals
GET /api/goals
PUT /api/goals/:id
DELETE /api/goals/:id
```

Progress

```
GET /api/progress
```

AI

```
POST /api/ai/insights
```

The exact endpoints should be adjusted according to the actual project requirements.

Page 16 — Validation and Error Handling

Input validation is necessary to prevent invalid data from entering the system.

For example, an activity duration should contain a valid numerical value.

Validation

The API can check:

- Required fields
- Data types
- Valid email format
- Valid dates
- Acceptable numerical values
- Authentication status

Error Responses

A consistent error format makes the API easier to use.

Example:

```
{
  "success": false,
  "message": "Invalid activity data"
}
```

HTTP Status Codes

Code	Meaning
200	Successful request
201	Resource created
400	Bad request
401	Unauthorized
403	Forbidden
404	Not found
500	Server error

Proper error handling improves reliability and debugging.

Page 17 — Security

Security is an important requirement for a fitness application because user accounts and personal information may be stored.

Important Security Measures

1. Hash passwords before storage.
2. Use authentication for protected endpoints.
3. Validate all user input.
4. Use authorization checks.
5. Avoid exposing sensitive information.
6. Use secure environment variables.
7. Handle errors without exposing internal system details.
8. Use HTTPS in production.
9. Apply appropriate rate limiting.
10. Keep dependencies updated.

Environment Variables

Secrets such as database credentials and API keys should not be hard-coded into source code.

Example:

```
DATABASE_URL=...  
JWT_SECRET=...  
AI_API_KEY=...
```

These values should be stored securely.

Page 18 — API Testing

Testing verifies whether the API works as expected.

A tool such as Postman can be used to test endpoints.

Testing Categories

Registration Test

Check whether a new user can register successfully.

Login Test

Check whether valid credentials return an authentication token.

Activity Test

Check whether activities can be created, retrieved, updated, and deleted.

Goal Test

Check whether users can create and manage goals.

Authentication Test

Check whether protected endpoints reject unauthorized requests.

Validation Test

Send invalid input and verify that the server returns an appropriate error.

Example Test Case

Test	Expected Result
Valid registration	User created
Duplicate email	Error returned
Valid login	Token returned
Invalid password	Authentication error

Test	Expected Result
Create activity	Activity stored
Invalid activity	Validation error

Page 19 — Advantages, Limitations and Future Enhancements

Advantages

The AI FitTrack API provides several advantages:

- Centralized fitness data.
- Reusable REST APIs.
- User-specific information.
- Structured activity tracking.
- Goal management.
- Progress monitoring.
- Possibility of AI-assisted insights.
- Separation between frontend and backend.
- Easy integration with web and mobile clients.

Limitations

Possible limitations include:

- AI-generated information may not always be accurate.
- The system depends on the quality of user-provided data.
- Authentication and security require careful implementation.
- External AI services may introduce additional dependencies.
- A basic implementation may not support wearable-device integration.

Future Enhancements

Future versions could include:

1. Wearable-device integration.
 2. Advanced analytics.
 3. Activity charts.
 4. Notification systems.
 5. More detailed dashboards.
 6. Voice-based interaction.
 7. Improved AI personalization.
 8. Cloud deployment.
 9. Automated reports.
 10. Integration with additional fitness services.
-

Page 20 — Conclusion

The **AI FitTrack API** is a backend project designed to provide structured fitness-tracking functionality through RESTful APIs. The system can manage users, profiles, exercises, activities, goals, and progress while providing a foundation for AI-assisted insights.

The project demonstrates important backend-development concepts including API design, authentication, database management, validation, error handling, security, testing, and service-layer architecture.

The API-based architecture also makes it possible for different client applications to communicate with the same backend. A web application, mobile application, or other client can consume the APIs without needing to directly interact with the database.

AI capabilities can further extend the system by analyzing user-provided activity information and generating useful summaries or general suggestions. However, AI-generated fitness information should be presented responsibly and should not replace professional medical or fitness advice.

Overall, the project provides a practical example of how backend technologies and AI-assisted functionality can be combined to create a modern fitness-tracking application.

References

1. Uploaded **Project Details – Backend** document, which lists AI FitTrack API under AI-Augmented Backend Development.
2. Project reference document supplied in the uploaded project list.
3. Project reference video listed in the uploaded project document.